ActPlane: OS-Enforced Agent Harnesses

Paper #XXX
Anonymous Submission

ACM Reference Format:
Paper #XXX. 2026. ActPlane: OS-Enforced Agent Harnesses. In *Proceedings of 2026 ACM SIGOPS Annual Technical Conference (ATC '26)*. ACM, New York, NY, USA, 2 pages.

Modern coding agents are governed largely by natural-language harnesses: `CLAUDE.md`, `AGENTS.md`, and similar files tell them to run tests before committing, avoid direct pushes, keep secrets local, use review tools, or ask for confirmation before destructive operations. These instructions are useful but probabilistic. A cooperative agent can forget them during a long task, or satisfy the wording at the tool layer while still reaching the same effect through `bash -c`, a Python subprocess, a helper binary, or an SDK. The result is a mismatch between the layer where policies are written and the layer where agent actions actually happen.

Existing defenses do not close this gap. Prompt-only constraints have no deterministic enforcement. Tool-layer guardrails such as policy checks over tool names and arguments are deterministic, but they only see effects that cross the guarded tool API. Whole-agent sandboxes draw a coarse perimeter, but cannot easily express workflow invariants such as "commit only after tests" or "this derived data may leave only after redaction." The OS boundary is the common layer below these routes: every exec, file access, and socket connect crosses a kernel interface regardless of how the agent initiated it.

ActPlane is an OS-level harness for AI agents. It compiles declarative source/sink/gate policies into an eBPF/BPF-LSM backend that maintains typed labels over processes, files, and endpoints. Labels propagate through fork/exec, file reads and writes, and connect-level endpoint edges. Rules then check whether a labeled subject may perform an action on a matching target. The policy language is intentionally agent-oriented: it expresses secret-derived egress, untrusted input requiring review, read-only sub-agents, mandatory mediation through a tool, and temporal gates such as testing before commit.

The central design choice is to treat agent policies as information-flow constraints rather than as tool-call filters. A source introduces a label, for example a process lineage started by `actplane run`, a repository secret, or a directory containing untrusted issue text. A sink names the operation to guard, for example an executable path, a file path, or a network endpoint. A gate records an event that justifies an otherwise forbidden flow, such as a redaction tool, a review step, a successful test run, or an explicit confirmation. A rule combines those objects with an explicit effect and a human-readable reason. This lets the policy say what must be true about the agent's execution history, not merely which high-level tool name appeared in a transcript.

The key semantic distinction is that ActPlane does not pretend every backend can implement every effect. `block` means pre-operation denial through a BPF-LSM hook returning `-EPERM`; it is not supported in tracepoint mode. `audit` reports while allowing the action to proceed. `kill` terminates the violating task and is useful for harness-level failure when pre-operation arguments are unavailable. Each rule declares its effect explicitly; there is no global fallback that silently converts `block` into `kill`.

This distinction matters for the paper's threat and harness model. ActPlane is not trying to prove that an arbitrary malicious process cannot escape every possible policy. It targets a common but underspecified workload: a powerful coding agent that is mostly cooperative, has long-running context, can call shells and helpers, and needs deterministic failure feedback when it crosses a project boundary. In that model, OS-level enforcement is useful even when the effect is harness-level termination rather than pre-operation denial. Killing a violating exec after `sched_process_exec`, for example, is not the same security primitive as LSM denial, but it is still an effective failed action from the agent's perspective and can be paired with a specific remediation.

ActPlane also closes the feedback loop. Kernel events contain only rule identifiers and enforcement metadata; the Rust runner maps those identifiers back to rule names, reasons, and remediation text, appends a model-facing payload to `.actplane/last-violation.txt`, and exposes a post-tool hook adapter that feeds new violations into the next agent turn. This keeps the kernel as the sole authority for matches while giving a cooperative agent enough semantic context to choose an allowed path.

The implementation follows this model with a deliberately small kernel interface. The BPF side normalizes hook-specific observations into a common event record containing subject identity, action kind, target descriptor, label masks, and the rule that fired. File and exec hooks perform target extraction; network hooks extract endpoint descriptors; process hooks maintain lineage. The common matcher then evaluates required and forbidden labels against compiled rule tables. User space owns YAML parsing, glob lowering, rule explanation, feedback-file management, and command launching. The default user experience is therefore a wrapper command such as `actplane run <agent-or-command>`, with an optional policy file when the current directory's policy should not be used.

The evaluation will answer four practical questions before claiming results. First, can ActPlane catch the same violation across direct tool calls, nested shell strings, subprocesses, SDK/helper paths, and direct syscalls? Second, does corrective feedback improve task completion and reduce repeated violations relative to bare `-EPERM` or `SIGKILL`? Third, what false positive rate arises from object-level taint, and do declassification and gate paths allow intended work? Fourth, what overhead does label propagation add to fork/exec, file, and connect events, and to real agent tasks? The evaluation matrix leaves numeric result cells TBD until the experiments are run.

The planned experiments separate mechanism from user experience. The bypass coverage experiment uses paired triggers that reach the same effect through different routes: a direct tool call, a shell string, a Python subprocess, a helper binary, an SDK path, and a direct syscall-oriented test where applicable. The feedback experiment compares prompt-only operation, audit-only reporting, failure without semantic feedback, and failure with ActPlane's feedback file and hook bridge. The false-positive experiment runs allowed workflows that should pass only after the correct gate, including redaction before egress, test-before-commit, confirmation-before-destructive-action, and read-only sub-agent delegation. The overhead experiment measures both per-event costs and whole-agent task time. Finally, the corpus experiment maps real `AGENTS.md`, `CLAUDE.md`, and similar instructions into observable, expressible, robust, and temptable categories before deriving a policy/task benchmark.

The mechanism is intentionally conservative. Kernel provenance systems such as CamQuery [3] already showed that cross-channel tracking and prevention are possible; agent-policy systems such as AgentSpec [5] and Progent [4] show the need for explicit runtime rules; and Fides [1] and CaMeL [2] show that information-flow abstractions are useful for agent safety. The paper's bet is that coding agents make this old capability newly useful at the OS boundary: the policy objects are project instructions, the subjects are agent lineages and sub-agents, the failures must be understandable to a model, and the useful definition of enforcement includes both pre-operation denial and deterministic harness feedback.

References

[1] Manuel Costa, Boris Koepf, Aashish Kolluri, Andrew Paverd, Mark Russinovich, Ahmed Salem, Shruti Tople, Lukas Wutschitz, and Santiago Zanella-Beguelin. 2025. Securing AI Agents with Information-Flow Control. arXiv:2505.23643. https://arxiv.org/abs/2505.23643

[2] Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramer. 2025. Defeating Prompt Injections by Design. arXiv:2503.18813. https://arxiv.org/abs/2503.18813

[3] Thomas F. J.-M. Pasquier, Xueyuan Han, Thomas Moyer, Adam Bates, Olivier Hermant, David Eyers, Jean Bacon, and Margo Seltzer. 2018. Runtime Analysis of Whole-System Provenance. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*. Association for Computing Machinery, Toronto, Canada, 1601–1616. doi:10.1145/3243734.3243776

[4] Tianneng Shi, Jingxuan He, Zhun Wang, Hongwei Li, Linyu Wu, Wenbo Guo, and Dawn Song. 2025. Progent: Programmable Privilege Control for LLM Agents. arXiv:2504.11703. https://arxiv.org/abs/2504.11703

[5] Haoyu Wang, Christopher M. Poskitt, and Jun Sun. 2025. AgentSpec: Customizable Runtime Enforcement for Safe and Reliable LLM Agents. arXiv:2503.18666. https://arxiv.org/abs/2503.18666